Dynamic Programming

Outline

- Introduction
- Fibonacci
- Matrix Chain-Product
- The General Technique
- 0-1 Knapsack Problem
- General approach
- Summary

Dynamic Programming

- Dynamic Programming is a general algorithm design technique. "Programming" here means "planning".
- Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems
- Main idea:
  - solve several smaller (overlapping) subproblems
  - record solutions in a table so that each subproblem is only solved once
  - final state of the table will be (or contain) solution

Optimization Problems

- **Optimization problems** have ***many solutions***.
- Each solution has a ***value*** (cost, gain, …)
- We want to compute an ***optimal solution*** (with minimal cost, maximal gain, …)
- **Examples:**
- There are many routes from Riyadh to Paris.
  - *We want to compute the shortest route, the most scenic route, the fastest route, …*
- There are many ways to schedule a set of jobs on multiple processors.
  - *Given different capabilities of the processors and different resource requirements of the jobs, we want to make sure that we can complete as many jobs as possible in as little time as possible.*

Greedy approach vs Dynamic programming

- Both solve optimization problems

Feature	Greedy Method	Dynamic Programming
Feasibility	Make whatever choice seems best at the moment in the hope that it will lead to global optimal solution.	Make decision at each step considering current problem and solution to previously solved sub problem to calculate optimal solution
Fashion	-Making the choices in a serial forward fashion, never looking back or revising previous choices. -Making the locally optimal choice at each stage.	-Computes the solution from bottom up by composing them from smaller optimal sub solutions. -Usually based on a recurrent formula that uses some previously calculated states. (Iteration)
Time complexity	faster	Generally slower

Divide & Conquer vs Dynamic programming

- Both divide the problem into smaller sub-problems

Divide & Conquer	**Dynamic Programming**
- Divide the problem into sub-problems, conquer each sub-problem recursively and combine these solutions. -Top-down approach.	-Solve the smaller sub-problems first, store them, look up to them whenever you need. -Bottom-up approach
Subproblems are independent of each other	Subproblems are interdependent (overlapping)

Fibonacci numbers

- The Fibonacci Sequence is the series of numbers:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

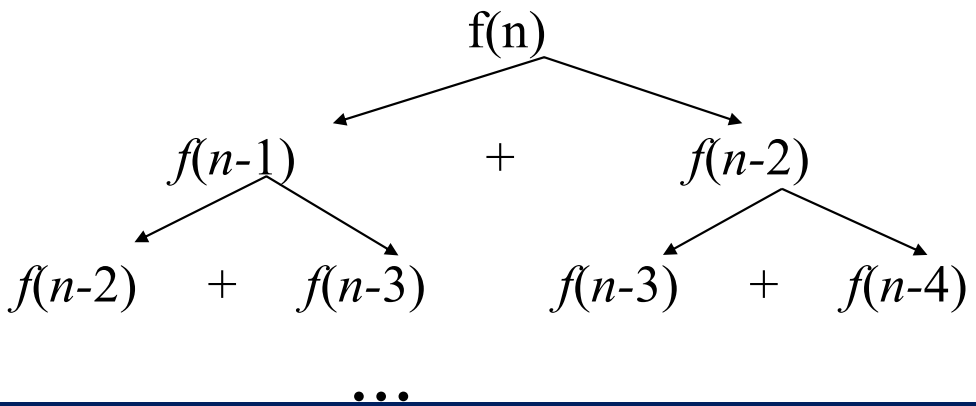
- Recall definition of Fibonacci numbers:

$f(0) = 0$

$f(1) = 1$

$f(n) = f(n\text{-}1) + f(n\text{-}2)$

- Computing the n^{th} Fibonacci number recursively (top-down):



Fibonacci numbers

► Algorithm 1.6 **nth Fibonacci Term (Recursive)**

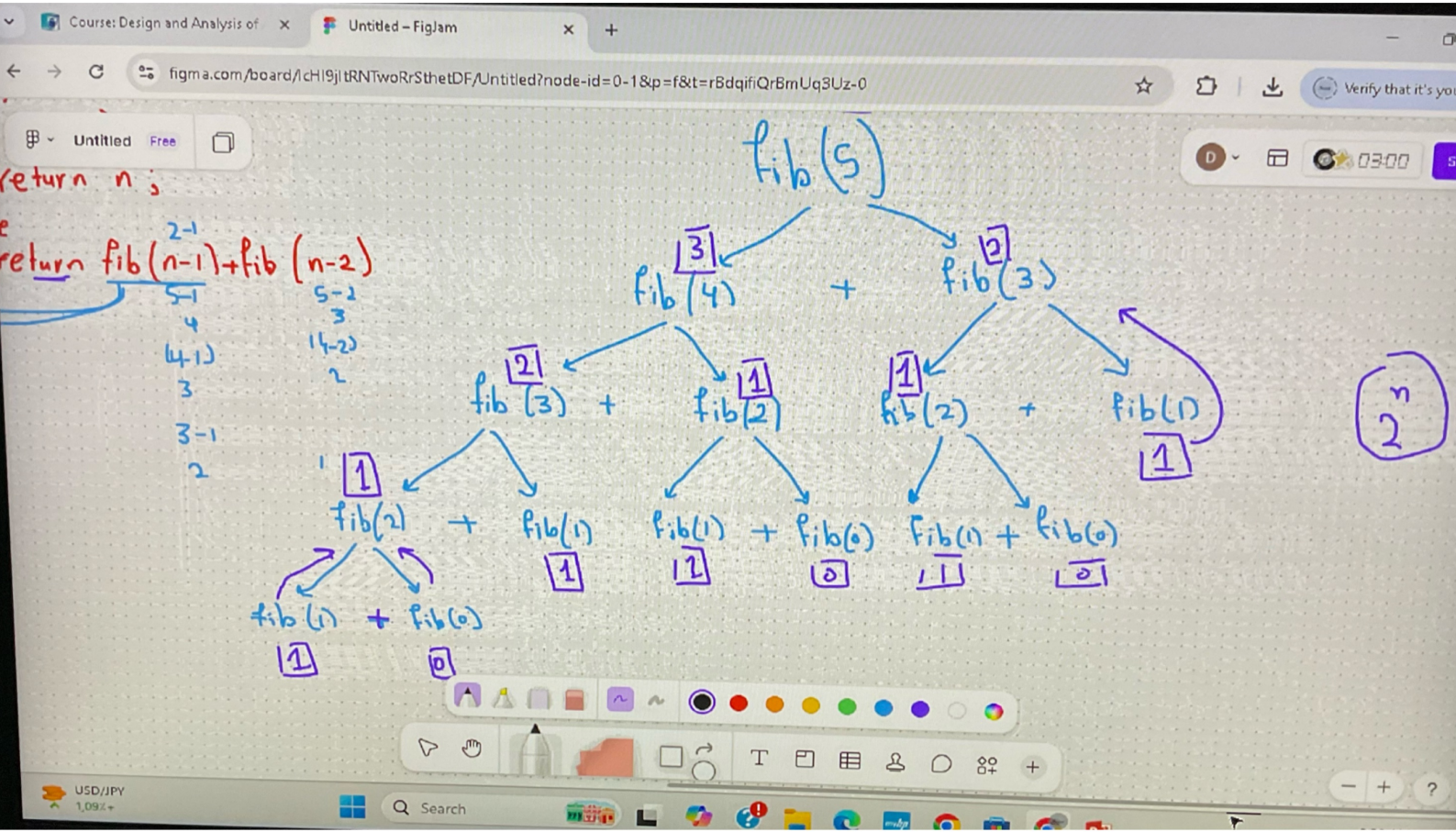
Problem: Determine the nth term in the Fibonacci sequence.

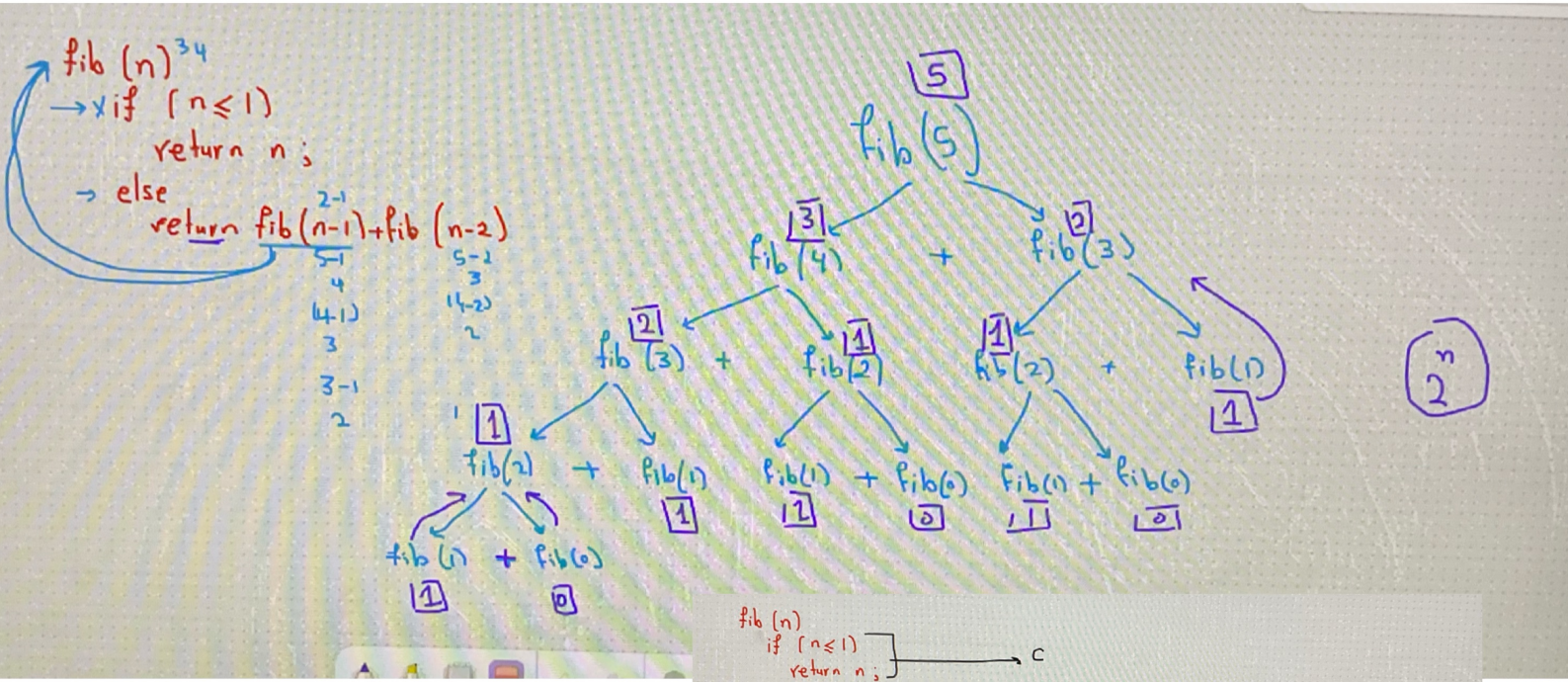
Inputs: a nonnegative integer n.

Outputs: *fib*, the nth term of the Fibonacci sequence.

```
int fib (int n)
{
    if (n <= 1)
        return n;
    else
        return fib(n-1) + fib(n-2);
}
```

- Time complexity: $T(n)=O(2^n)$

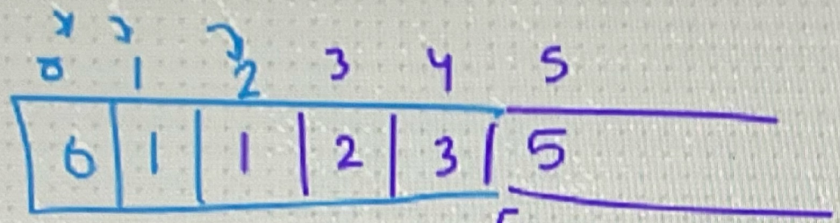

Course: Design and Analysis of
Untitled – FigJam
figma.com/board/lcHI9jltRNTwoRrSthetDF/Untitled?node-id=0-1&p=f&t=rBdqifiQrBmUq3Uz-0
Untitled
Free
return n ;
return fib(n-1)+fib(n-2)
fib(5)
fib(4)
fib(3)
fib(3)
fib(2)
fib(2)
fib(1)
fib(2)
fib(1)
fib(1)
fib(0)
fib(1)
fib(0)
fib(1)
fib(0)
2^n
Search
USD/JPY



fib (n)
if (n≤1) → c
return n;
else
return fib(n-1)+fib(n-2) → $T(n-1) + T(n-1)$

$\approx 2T(n-1)$

$C + 2T(n-1) \longrightarrow 2^n$



linear_fib (n) {

Array F [0, n]

F[0] = 0

F[1] = 1

for (i=2 ; i < n ; i++)

F[i] = F[i-1] + F[i-2]

}

Example: Fibonacci numbers

Computing the n^{th} fibonacci number using bottom-up iteration:

- $f(0) = 0$
- $f(1) = 1$
- $f(2) = 0+1 = 1$
- $f(3) = 1+1 = 2$
- $f(4) = 1+2 = 3$
- $f(5) = 2+3 = 5$
-
-
-
- $f(n-2) =$
- $f(n-1) =$
- $f(n) = f(n-1) + f(n-2)$

Fibonacci numbers

► Algorithm 1.7 **nth Fibonacci Term (Iterative)**

Problem: Determine the nth term in the Fibonacci sequence.

Inputs: a nonnegative integer n.

Outputs: *fib2*, the nth term in the Fibonacci sequence.

```
int fib2 (int n)
{
   index i;
   int f[0..n];

   f[0]=0;
   if (n > 0)
      f[1]=1;
      for (i=2; i<=n; i++)
         f[i] = f[i-1] + f[i-2];
   }
   return f[n];
}
```

- Time complexity: T(n)=O(n)

Fibonacci numbers

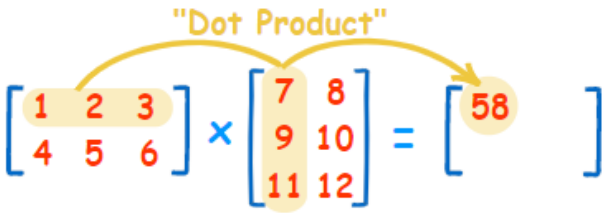
- Table 1.2 A comparison of Algorithms 1.6 and 1.7

n	$n+1$	$2^{n/2}$	Execution Time Using Algorithm 1.7	Lower Bound on Execution Time Using Algorithm 1.6
40	41	1,048,576	41 ns*	1048 $\mu s^{\dagger}$
60	61	1.1×10^9	61 ns	1 s
80	81	1.1×10^{12}	81 ns	18 min
100	101	1.1×10^{15}	101 ns	13 days
120	121	1.2×10^{18}	121 ns	36 years
160	161	1.2×10^{24}	161 ns	3.8×10^7 years
200	201	1.3×10^{30}	201 ns	4×10^{13} years

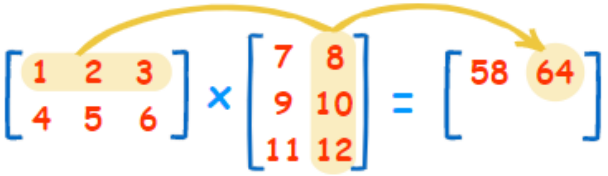
*1 ns $= 10^{-9}$ second.

$^{\dagger}1\ \mu s = 10^{-6}$ second.

matrix multiplication



- The "Dot Product" is where we **multiply matching members**, then sum up:
- (1, 2, 3) • (7, 9, 11) = 1×7 + 2×9 + 3×11
  = 58

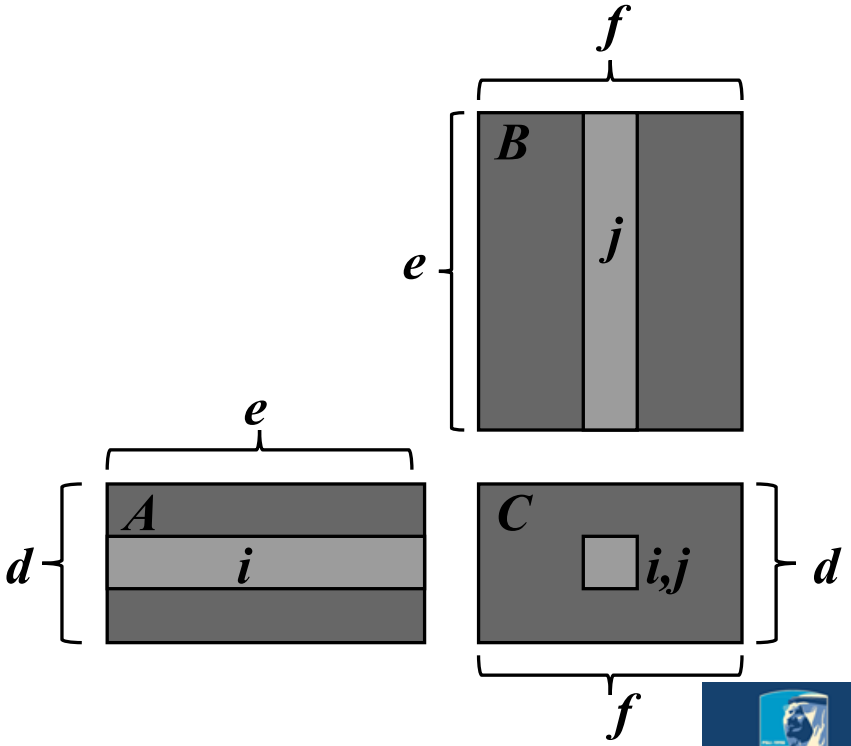


- (1, 2, 3) • (8, 10, 12) = 1×8 + 2×10 + 3×12
  = 64

Matrix Chain-Products

- **Matrix Chain-Products**
- Review: Matrix Multiplication.
  - **C** = **A*****B**
  - **A** *is* **d** × **e** *and* **B** *is* **e** × **f**
  - **O(d·e·f)** *time*

$$C[i,j] = \sum_{k=0}^{e-1} A[i,k] * B[k,j]$$



Matrix Chain-Products *

- **Matrix Chain-Product:**
  - *Compute $A=A_0*A_1*\ldots*A_{n-1}$*
  - *A_i is $d_i \times d_{i+1}$*
  - *Problem: How to parenthesize?*
- Example
  - *B is 3 × 100*
  - *C is 100 × 5*
  - *D is 5 × 5*
- Find the total number of operations for:
  - *(B*C)*D takes 1500 + 75 = 1575 ops*
  - *B*(C*D) takes 1500 + 2500 = 4000 ops*
- Goal: Find the optimal way to multiply given matrices to perform the fewest multiplications

Enumeration Approach (Naïve approach)

- **Matrix Chain-Product Alg.:**
  - *Try all possible ways to parenthesize* $A=A_0*A_1*\ldots*A_{n-1}$
  - *Calculate number of ops for each one*
  - *Pick the one that is best*
- Running time:
  - *The number of parenthesizations is equal to the number of binary trees with n nodes*
  - *This is* ***exponential****!*
  - *it is almost* 4^n*.*
  - *This is a terrible algorithm!*

Greedy Approach

- Idea #1: repeatedly select the product that uses (up) the most operations.
- Counter-example:
  - *A is 10 × 5*
  - *B is 5 × 10*
  - *C is 10 × 5*
  - *D is 5 × 10*
  - *Greedy idea #1 gives (A*B)*(C*D), which takes 500+1000+500 = 2000 ops*
  - *A*((B*C)*D) takes 500+250+250 = 1000 ops*

Another Greedy Approach

- Idea #2: repeatedly select the product that uses the fewest operations.
- Counter-example:
  - *A is 101 × 11*
  - *B is 11 × 9*
  - *C is 9 × 100*
  - *D is 100 × 99*
  - *Greedy idea #2 gives A*((B*C)*D)), which takes 109989+9900+108900=228789 ops*
  - *(A*B)*(C*D) takes 9999+89991+89100=189090 ops*
- The greedy approach is not giving us the optimal value.

how we shud paranthesize:

A_1	A_2	A_3	A_4
5×4	4×6	6×2	2×7

Step 1: $m[1,2]$ $(A_1 \times A_2)$ $5 \times 4 \times 6$ | $m[2,3]$ $A_2\ A_3$ $4 \times 6 \times 2$ | $m[3,4]$ $A_3\ A_4$ $6 \times 2 \times 7$

m

	1	2	3	4
1	0	120	88	158
2		0	48	104
3			0	84
4				0

store cost # of mult

Step 2: $(A_1\ A_2)\ A_3$ or $A_2\ A_3\ A_4$

$m[1,2]$ $m[2,3]$

$(A_1\ A_2)\ A_3$ or $A_1(A_2\ A_3)$ | $(A_2\ A_3)\ A_4$ | $A_2(A_3\ A_4)$

5×6 6×7 | 5×4 4×2 | 4×6 6×2 | 4×6 6×7

$120 + 5 \times 6 \times 2$ | $5 \times 4 \times 2 + 48$ | $48 + 4 \times 2 \times 7$ | $4 \times 6 \times 7 + 84$

$120 + 60$ | $40 + 48$ | $48 + 56$ | $168 + 84$

$= 180$ | 88 | 104 | 252

s

	1	2	3	4
1	0	1	1	3
2		0	2	3
3			0	3
4				0

store where to split

Step: 3

A_1 A_2 A_3 A_4

5×2 2×7 | 5×6 6×2 | 5×4 4×7

$(A_1\ A_2\ A_3)(A_4)$ | $(A_1\ A_2)(A_3\ A_4)$ | $(A_1)(A_2\ A_3\ A_4)$

$m[1,3]$

$88 + 5 \times 2 \times 7$

$88 + 70$

158

$120 + 5 \times 6 \times 7 + 84$

$120 + 210 + 84$

XX

$m(2,4)$

$5 \times 4 \times 7 + 104$

$140 + 104$

244

"Recursive" Approach *

- Define **subproblems**:
  - *Find the best parenthesization of $A_i*A_{i+1}*\ldots*A_j$.*
  - *Let $N_{i,j}$ denote the number of operations done by this subproblem.*
  - *The optimal solution for the whole problem is $N_{0,n-1}$.*
- **Subproblem optimality**: The optimal solution can be defined in terms of optimal subproblems
  - *There has to be a final multiplication (root of the expression tree) for the optimal solution.*
  - *Say, the final multiply is at index i: $(A_0*\ldots*A_i)*(A_{i+1}*\ldots*A_{n-1})$.*
  - *Then the optimal solution $N_{0,n-1}$ is the sum of two optimal subproblems, $N_{0,i}$ and $N_{i+1,n-1}$ plus the time for the last multiply.*

Characterizing Equation

- The global optimal has to be defined in terms of optimal subproblems, depending on where the final multiply is at.
- Let us consider all possible places for that final multiply:
  - *Recall that A_i is a $d_i \times d_{i+1}$ dimensional matrix.*
  - *So, a characterizing equation for $N_{i,j}$ is the following:*

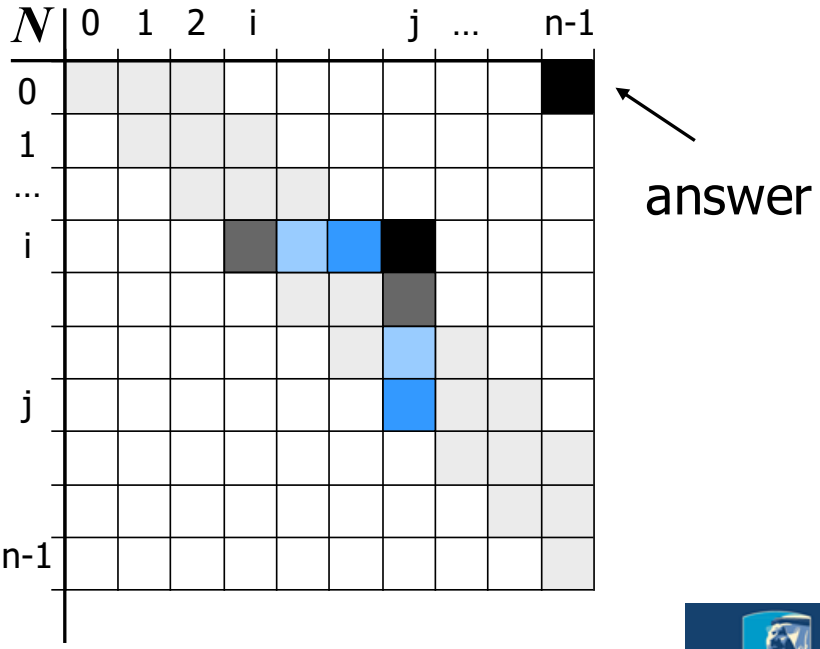
$$N_{i,j} = \min_{i \le k < j} \{ N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1} \}$$

- Note that subproblems are not independent–the **subproblems overlap**.

Dynamic Programming Algorithm Visualization

- The bottom-up construction fills in the N array by diagonals
- $N_{i,j}$ gets values from previous entries in i-th row and j-th column
- Filling in each entry in the N table takes O(n) time.
- Total run time: $O(n^3)$
- Getting actual parenthesization can be done by remembering "k" for each N entry

$$N_{i,j} = \min_{i \le k < j} \{ N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1} \}$$



Dynamic Programming Algorithm

- Since subproblems overlap, we don't use recursion.
- Instead, we construct optimal subproblems "bottom-up."
- $N_{i,i}$'s are easy, so start with them
- Then do problems of "length" 2,3,… subproblems, and so on.
- Running time: $O(n^3)$

Algorithm ***matrixChain***(S):

Input: sequence S of n matrices to be multiplied

Output: number of operations in an optimal parenthesization of S

for $i \leftarrow 1$ **to** $n - 1$ **do**
    $N_{i,i} \leftarrow 0$

add 0 to all cells

for $b \leftarrow 1$ **to** $n - 1$ **do**
    //{ $b = j - i$ is the length of the problem }
    for $i \leftarrow 0$ **to** $n - b - 1$ **do**
        $j \leftarrow i + b$
        $N_{i,j} \leftarrow +\infty$
        for $k \leftarrow i$ **to** $j - 1$ **do**
            $N_{i,j} \leftarrow \min\{N_{i,j}, N_{i,k} + N_{k+1,j} + d_i d_{k+1} d_{j+1}\}$

calculate

return $N_{0,n-1}$

figma.com/board/ ... Ask Google Finish update
Untitled Free
D 03:00 Share

A_1 A_2 A_3

2×3 3×2 4×2

d_0 d_1 d_2 d_3

$$c[i,j] = \min\left\{ c[i,k] + c[k+1,j] + d_{i-1} \times d_k \times d_j \right\} \quad i \le k < j$$

$c[1,3] =$

Value1 = cost = $c[1,1] + c[2,3] + d_0 \times d_1 \times d_3$

i k

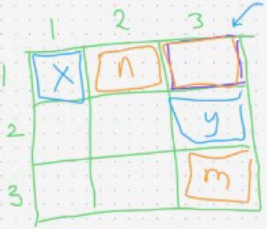
$x + y + 2 \times 3 \times 2 =$

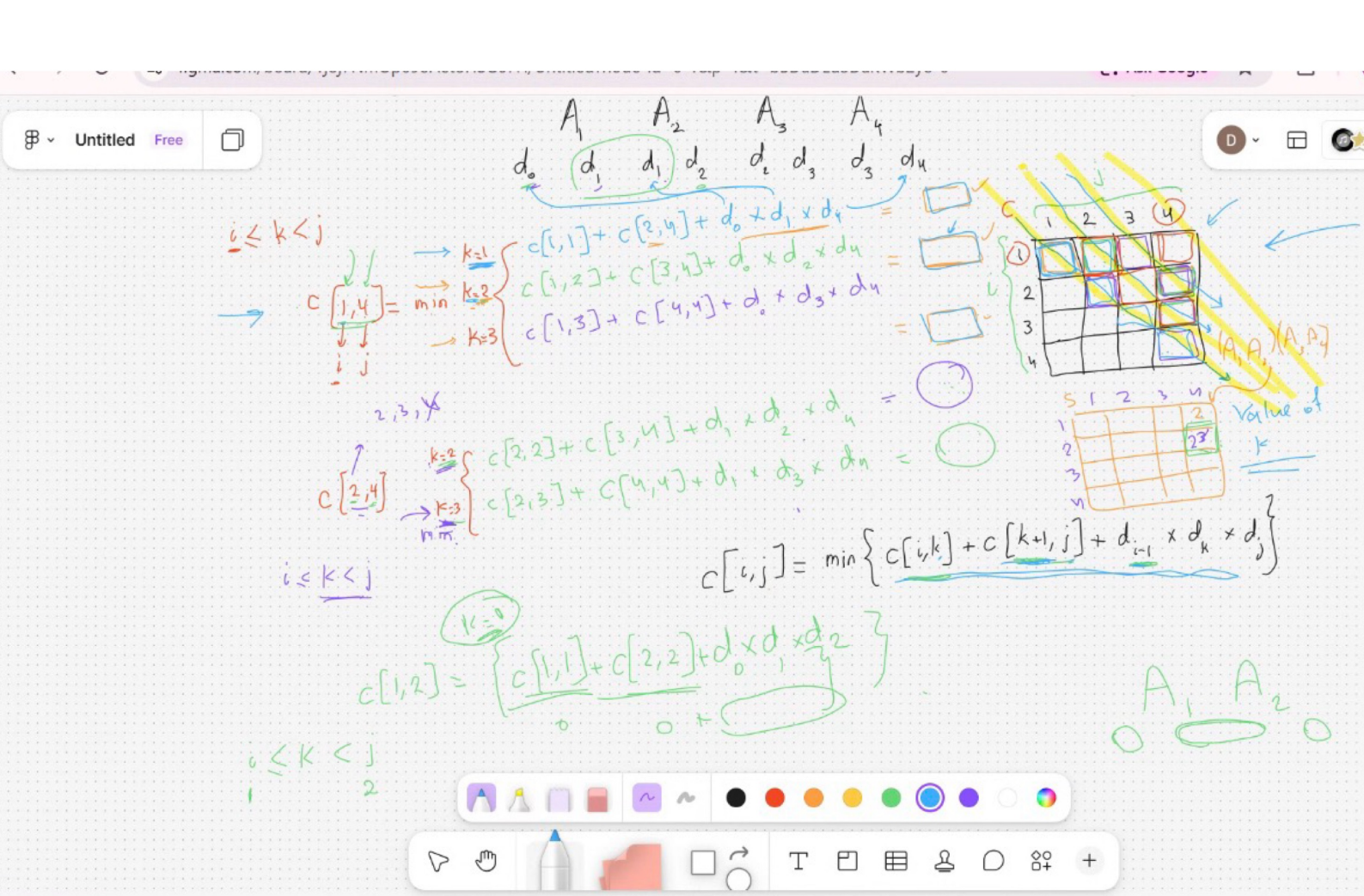
increment k

Value2 = Cost = $c[1,2] + c[3,3] + d_0 \times d_2 \times d_3$

i k

$n + m + 2 \times 4 \times 2 =$




Untitled
Free
A1 A2 A3 A4
d0 d1 d2 d3 d4
i ≤ k < j
c[1,4] = min
k=1
k=2
k=3
c[1,1] + c[2,4] + d0 x d1 x d4
c[1,2] + c[3,4] + d0 x d2 x d4
c[1,3] + c[4,4] + d0 x d3 x d4
c[2,4]
c[2,2] + c[3,4] + d1 x d2 x d4
c[2,3] + c[4,4] + d1 x d3 x d4
c[i,j] = min { c[i,k] + c[k+1,j] + di-1 x dk x dj }
c[1,2] = { c[1,1] + c[2,2] + d0 x d1 x d2 }
Value of k

Example

- The algorithm takes $O(n^3)$ time and requires $O(n^2)$ space.
- Example:
- Consider the following six matrix problem.

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

p_0 p_1 p_1 p_2 p_2 p_3 p_3 p_4 p_4 p_5 p_5 p_6

- The problem therefore can be phrased as one of filling in the following table representing the values m.

i\j	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

Example (Contd.)

- Chains of length 2 are easy, as there is no minimization required, so
- $m[i, i+1] = p_{i-1}p_ip_{i+1}$

- $m[1, 2] = 10x20x5 = 1000$
- $m[2, 3] = 20x5x15 = 1500$
- $m[3, 4] = 5x15x50 = 3750$
- $m[4, 5] = 15x50x10 = 7500$
- $m[5, 6] = 50x10x15 = 7500$

i\j	1	2	3	4	5	6
1	0	1000				
2		0	1500			
3			0	3750		
4				0	7500	
5					0	7500
6						0

Example (Contd.)

- Chains of length 3 require some minimization – but only one each.

- m[1,3]=min{(m[1,1]+m[2,3]+$p_0p_1p_3$),(m[1,2]+m[3,3]+$p_0p_2p_3$)}
  = min{(0+1500+10x20x15), (1000+0+10x5x15)}
  = min { 4500, 1750 } = 1750
- m[2,4]=min{(m[2,2]+m[3,4]+$p_1p_2p_4$),(m[2,3]+m[4,4]+$p_1p_3p_4$)}
  = min{(0+3750+20x5x50), (1500+0+20x15x50)}
  = min { 8750, 16500 } = 8750
- m[3,5]=min{(m[3,3]+m[4,5]+$p_2p_3p_5$),(m[3,4]+m[5,5]+$p_2p_4p_5$)}
  = min{(0+7500+5x15x10), (3750+0+5x50x10)}
  = min { 8250, 6250 } = 6250
- m[4,6]=min{(m[4,4]+m[5,6]+$p_3p_4p_6$),(m[4,5]+m[6,6]+$p_3p_5p_6$)}
  = min{(0+7500+15x50x15), (7500+0+15x10x15)}
  = min { 18750, 9750 } = 9750

i\j	1	2	3	4	5	6
1	0	1000	1750			
2		0	1500	8750		
3			0	3750	6250	
4				0	7500	9750
5					0	7500
6						0

Example (Contd.)

- $m[1,4]=\min\{(m[1,1]+m[2,4]+p_0p_1p_4),(m[1,2]+m[3,4]+p_0p_2p_4),$
  $(m[1,3]+m[4,4]+p_0p_3p_4)\}$
  $= \min\{(0+8750+10x20x50), (1000+3750+10x5x50),$
  $(1750+0+10x15x50)\}$
  $= \min\{ 18750, 7250, 9250 \} = 7250$
- $m[2,5]=\min\{(m[2,2]+m[3,5]+p_1p_2p_5),(m[2,3]+m[4,5]+p_1p_3p_5),$
  $(m[2,4]+m[5,5]+p_1p_4p_5)\}$
  $= \min\{(0+6250+20x5x10), (1500+7500+20x15x10),$
  $(8750+0+20x50x10)\}$
  $= \min\{ 7250, 12000, 18750 \} = 7250$
- $m[3,6]=\min\{(m[3,3]+m[4,6]+p_2p_3p_6),(m[3,4]+m[5,6]+p_2p_4p_6),$
  $(m[3,5]+m[6,6]+p_2p_5p_6)\}$
  $= \min\{(0+9750+5x15x15), (3750+7500+5x50x15),$
  $(6250+0+5x10x15)\}$
  $= \min\{ 10875, 15000, 7000 \} = 7000$

i\j	1	2	3	4	5	6
1	0	1000	1750	7250		
2		0	1500	8750	7250	
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Example (Contd.)

- $m[1,5]=\min\{(m[1,1]+m[2,5]+p_0p_1p_5),(m[1,2]+m[3,5]+p_0p_2p_5),$
  $(m[1,3]+m[4,5]+p_0p_3p_5),(m[1,4]+m[5,5]+p_0p_4p_5)\}$
  $= \min\{(0+7250+10x20x10), (1000+6250+10x5x10),$
  $(1750+7500+10x15x10), (7250+0+10x50x10)\}$
  $= \min\{ 9250, 7750, 10750, 12250 \} = 7750$
- $m[2,6]=\min\{(m[2,2]+m[3,6]+p_1p_2p_6),(m[2,3]+m[4,6]+p_1p_3p_6),$
  $(m[2,4]+m[5,6]+p_1p_4p_6),(m[2,5]+m[6,6]+p_1p_5p_6)\}$
  $= \min\{(0+7000+20x5x15), (1500+9750+20x15x15),$
  $(8750+7500+20x50x15), (7250+0+20x10x15)\}$
  $= \min\{ 8500, 15750, 31,250, 10250 \} = 8500$
- $m[1,6]=\min\{(m[1,1]+m[2,6]+p_0p_1p_6),(m[1,2]+m[3,6]+p_0p_2p_6),$
  $(m[1,3]+m[4,6]+p_0p_3p_6),(m[1,4]+m[5,6]+p_0p_4p_6), (m[1,5]+m[6,6]+p_0p_5p_6)\}$
  $= \min\{(11500, 8750, 13750, 22250, 9250 \} = 8750$

i\j	1	2	3	4	5	6
1	0	1000	1750	7250	7750	8750
2		0	1500	8750	7250	8500
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Dynamic Programming Approach (Contd.)

- The algorithm takes $O(n^3)$ time and requires $O(n^2)$ space.
- Step 4: Constructing an optimal solution
  - Our algorithm computes the minimum-cost table *m* and the split table *s.*
  - The optimal solution can be constructed from the split table *s.*
  - Each entry $s[i, j]=k$ shows where to split the product $A_i A_{i+1} \ldots A_j$ for the minimum cost.
  - The following recursive procedure prints an optimal parenthesization.

```
Algorithm PrintOptimalPerens(s, i, j)
{       if (i=j) then
              Print "A"i;
        else
        {     Print "(";
              PrintOptimalPerens(s, i, s[i,j]);
              PrintOptimalPerens(s, s[i,j]+1, j);
              Print ")";
        }
}
```

Example (Contd.)

- So far we have decided that the best way to parenthesize the expression results in 8750 multiplication.
- But we have not addressed how we should actually DO the multiplication to achieve the value.
- However, look at the last computation we did – the minimum value came from computing

$$A = (A_1A_2)(A_3A_4A_5A_6)$$

- Therefore in an auxiliary array, we store value s[1,6]=2.
- In general, as we proceed with the algorithm, if we find that the best way to compute $A_{i..j}$ is as

$$A_{i..j} = A_{i..k}A_{(k+1)..j}$$

then we set

$$s[i, j] = k.$$

- Then from the values of k we can reconstruct the optimal way to parenthesize the expression.

Example (Contd.)

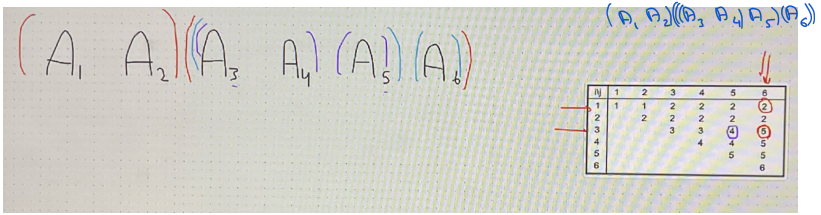
- If we do this then we find that the s array looks like this:

i\j	1	2	3	4	5	6
1	1	1	2	2	2	2
2		2	2	2	2	2
3			3	3	4	5
4				4	4	5
5					5	5
6						6

- We already know that we must compute $A_{1..2}$ and $A_{3..6}$.
- By looking at s[3,6] = 5, we discover that we should compute $A_{3..6}$ as $A_{3..5}A_{6..6}$ and then by seeing that s[3,5] = 4, we get the final parenthesization

$$A = ((A_1A_2)(((A_3A_4)A_5)A_6)).$$

- And quick check reveals that this indeed takes the required 8750 multiplications.



Another Example

Array dimensions:

- A_1: 2 x 3
- A_2: 3 x 5
- A_3: 5 x 2
- A_4: 2 x 4
- A_5: 4 x 3

Array sizes: $A_i = d_{i-1}$ by d_i :

j	0	1	2	3	4	5
d_j	2	3	5	2	4	3

i, j	1	2	3	4	5
1	0	30	42	58	78
2		0	30	54	72
3			0	40	54
4				0	24
5					0

Calculating M_{25} = number of multiplies in the optimal way to parenthesize $A_2A_3A_4A_5$:

$$M_{25} = \min \begin{cases} M_{22} + M_{35} + d_1d_2d_5, \\ M_{23} + M_{45} + d_1d_3d_5, \\ M_{24} + M_{55} + d_1d_4d_5 \end{cases} = \min \begin{cases} 0 + 54 + 45 = 99 \\ 30 + 24 + 18 = 72 \\ 54 + 0 + 36 = 90 \end{cases} = \min \begin{cases} (A_2)(A_3A_4A_5) \\ (A_2A_3)(A_4A_5) \\ (A_2A_3A_4)(A_5) \end{cases}$$

Untitled – FigJam

figma.com/board/hbt1LwMcRtptcCNIVTQKVG/Untitled?t=q1vWM0EkJoakCWyf-0

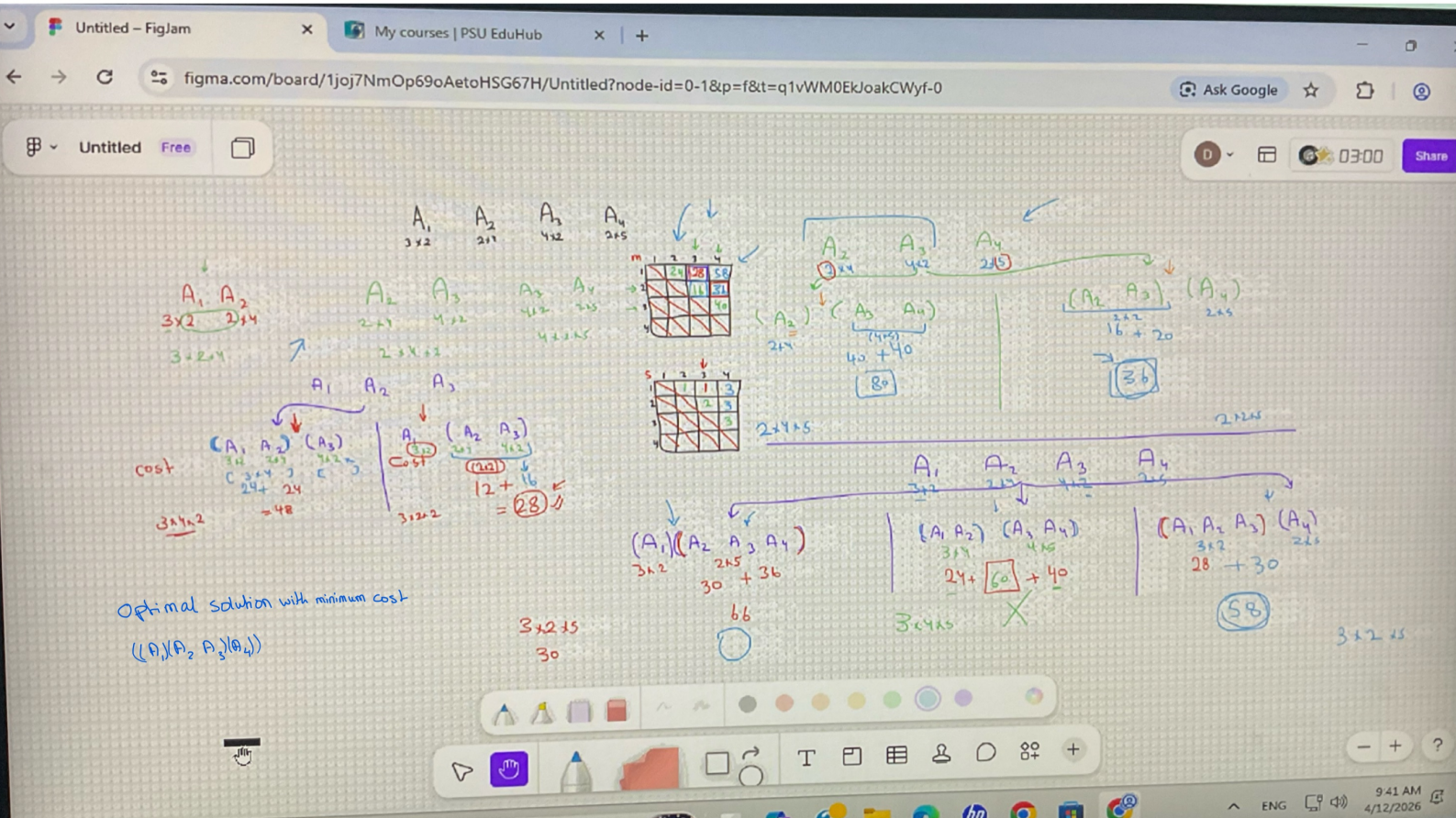
Untitled Free

1] Draw the state space generated by B&B.

2] What are the items with hieghest profit.

3] What is total profit for the optimal solution.

i	P_i	w_i	P_i/w_i
1	16	5	
2	22	7	
3	12	4	
4	8	3	



$$C[i,j] = \min\{C[i,k] + C[k+1,j] + d_{i-1} \times d_k \times d_j\}$$

The General Dynamic Programming Technique *

- Applies to a problem that at first seems to require a lot of time (possibly exponential), provided we have:
  - ***Simple subproblems:*** *the subproblems can be defined in terms of a few variables, such as j, k, l, m, and so on.*
  - ***Subproblem optimality:*** *the global optimum value can be defined in terms of optimal subproblems*
  - ***Subproblem overlap:*** *the subproblems are not independent, but instead they overlap (hence, should be constructed bottom-up).*

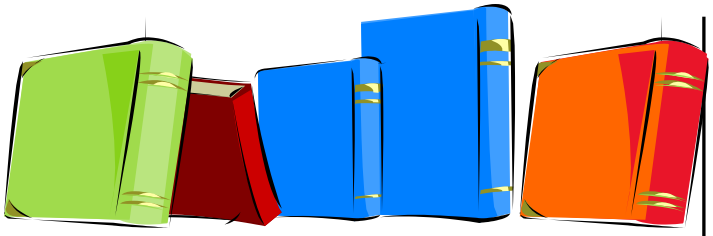
The 0/1 Knapsack Problem

- Given: A set S of n items, with each item i having
  - w_i - *a positive weight*
  - b_i - *a positive benefit*
- Goal: Choose items with maximum total benefit but with weight at most W.
- If we are **not** allowed to take fractional amounts, then this is the **0/1 knapsack problem**.
  - *In this case, we let T denote the set of items we take*
  - *Objective: maximize* $\sum_{i \in T} b_i x_i$
  - *Constraint:* $\sum_{i \in T} w_i x_i \leq W$
  - X_i = *0 or 1*

Example

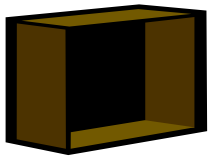
- Given: A set S of n items, with each item i having
  - b_i - *a positive "benefit"*
  - w_i - *a positive "weight"*
- Goal: Choose items with maximum total benefit but with weight at most W.

Items:



	1	2	3	4	5
Weight:	4 in	2 in	2 in	6 in	2 in
Benefit:	20 SR	3 SR	6 SR	25 SR	80 SR

"knapsack"



box of width 9 in

Solution:

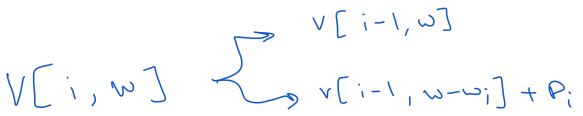
- item 5 (80 SR, 2 in)
- item 3 (6 SR, 2in)
- item 1 (20 SR, 4in)

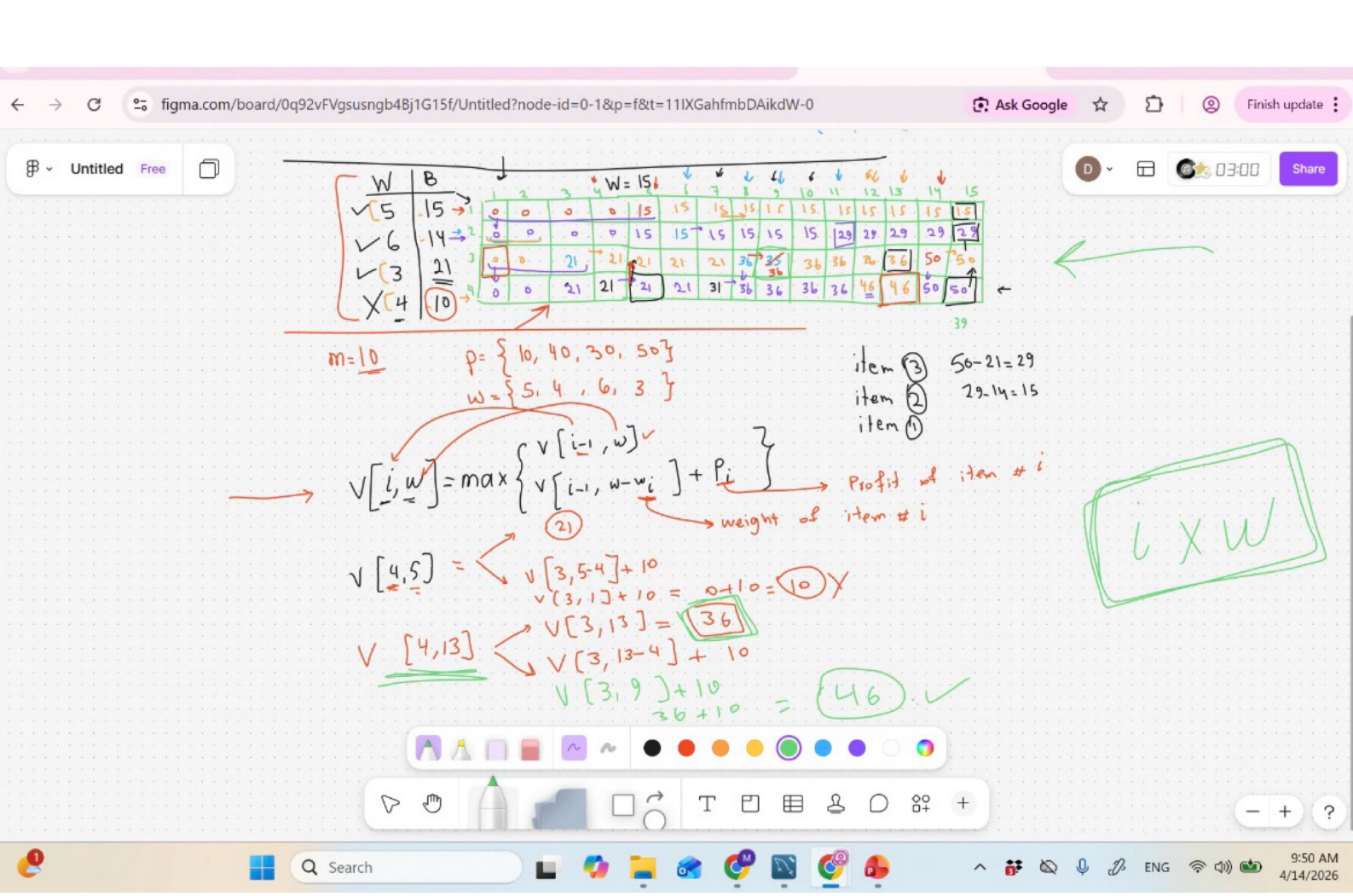
A 0/1 Knapsack Algorithm,

- S_k: Set of items numbered 1 to k.
- Define B[k,w] to be the best selection from S_k with weight at most w
- Good news: this does have subproblem optimality.

$$B[k,w] = \begin{cases} B[k-1,w] & \text{if } w_k > w \\ \max\{B[k-1,w],\ B[k-1,w-w_k]+b_k\} & \text{else} \end{cases}$$

- I.e., the best subset of S_k with weight at most w is either
  - *the best subset of S_{k-1} with weight at most w or*
  - *the best subset of S_{k-1} with weight at most $w-w_k$ plus item k*




figma.com/board/0q92vFVgsusngb4Bj1G15f/Untitled?node-id=0-1&p=f&t=11IXGahfmbDAikdW-0
Ask Google
Finish update
Untitled
Free
Share
W
B
5
15
6
14
3
21
4
10
W = 15
m=10
p= {10, 40, 30, 50}
w= {5, 4, 6, 3}
item 3
item 2
item 1
50-21=29
29-14=15
V[i,w] = max { V[i-1, w] , V[i-1, w-w_i] + P_i }
Profit of item # i
weight of item # i
i x w
V[4,5] = V[3,5-4] + 10
V[3,1] + 10 = 0+10 = 10
V[4,13] = V[3,13] = 36
V[3,13-4] + 10
V[3,9] + 10
36 + 10 = 46
Search
ENG
9:50 AM
4/14/2026

Recursive Formula for subproblems

- Recursive formula for subproblems:

$$B[k,w] = \begin{cases} B[k-1,w] & \text{if } w_k > w \\ \max\{B[k-1,w], B[k-1,w-w_k]+b_k\} & \text{else} \end{cases}$$

- It means, that the best subset of S_k that has total weight w is one of the two:

1) the best subset of S_{k-1} that has total weight w, **or**
2) the best subset of S_{k-1} that has total weight w-w_k plus the item k

Recursive Formula

$$B[k,w] = \begin{cases} B[k-1,w] & \text{if } w_k > w \\ \max\{B[k-1,w], B[k-1,w-w_k]+b_k\} & \text{else} \end{cases}$$

- The best subset of S_k that has the total weight w, either contains item k or not.
- First case: $w_k > w$. Item k can't be part of the solution, since if it was, the total weight would be $> w$, which is unacceptable
- Second case: $w_k <= w$. Then the item k can be in the solution, and we choose the case with greater value

0-1 Knapsack Algorithm

```
for w = 0 to W
    B[0,w] = 0
for i = 0 to n
    B[i,0] = 0
for i = 1 to n
    for w = 0 to W
        if w_i <= w // item i can be part of the solution
            if b_i + B[i-1,w-w_i] > B[i-1,w]
                B[i,w] = b_i + B[i-1,w- w_i]
            else
                B[i,w] = B[i-1,w]
        else B[i,w] = B[i-1,w]  // w_i > w
```

Running time

```
for w = 0 to W
    B[0,w] = 0
for i = 0 to n
    B[i,0] = 0
for i = 1 to n
    for w = 0 to W
        < the rest of the code >
```

$O(W)$

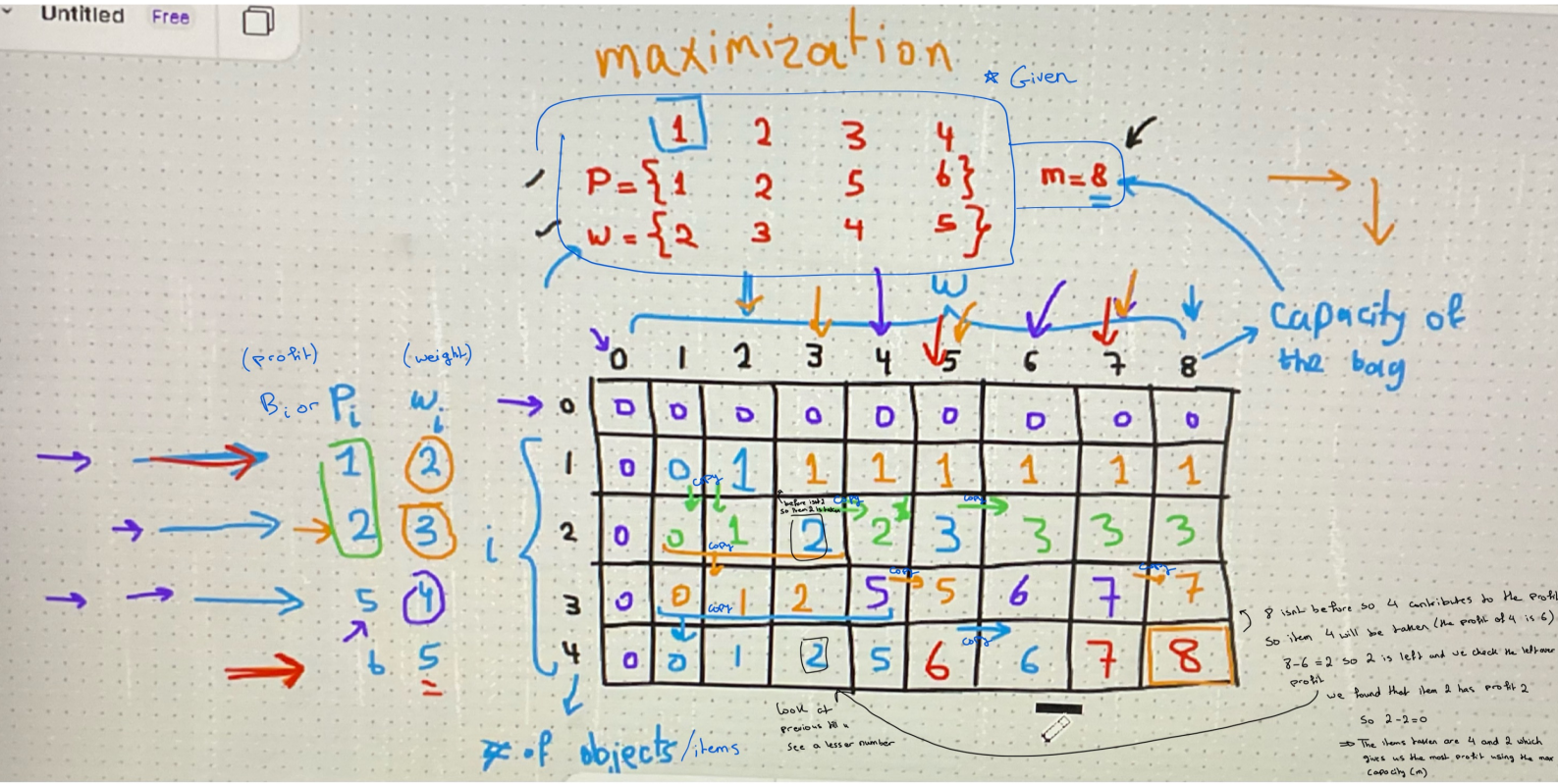
Repeat n times

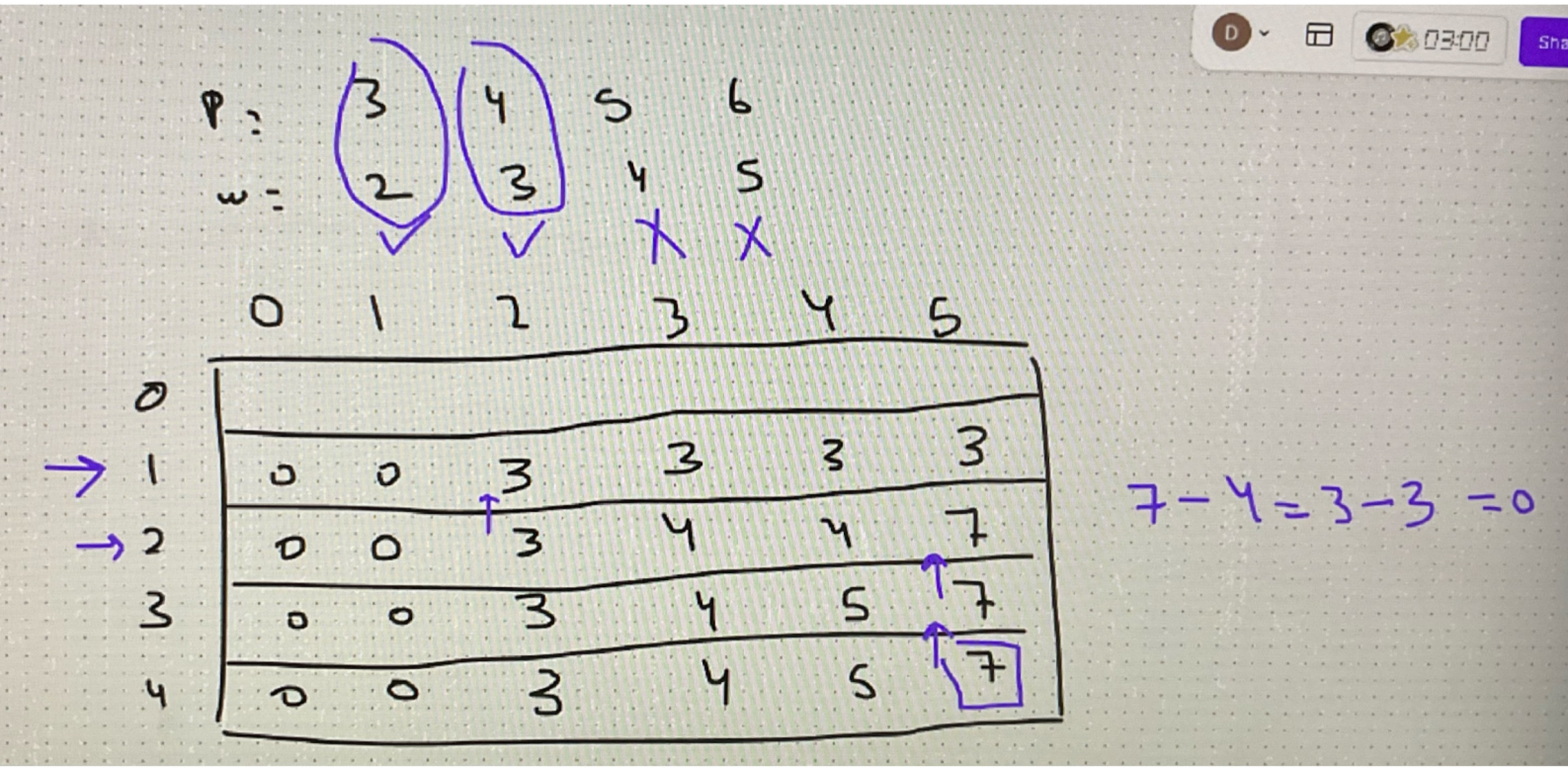
$O(W)$

What is the running time of this algorithm?

$O(n*W)$

Remember that the brute-force algorithm takes $O(2^n)$


Untitled
Free
maximization
Given
P = {1 2 5 6}
w = {2 3 4 5}
m = 8
capacity of the bag
(profit)
(weight)
B_i or P_i
w_i
i
# of objects/items
Look at previous row & see a lesser number
8 is not before so 4 contributes to the profit
So item 4 will be taken (the profit of 4 is 6)
8-6 = 2 so 2 is left and we check the leftover profit
we found that item 2 has profit 2
So 2-2=0
⇒ The items taken are 4 and 2 which gives us the most profit using the max capacity (m)


P: 3 4 5 6
w: 2 3 4 5
0 1 2 3 4 5
0
1 0 0 3 3 3 3
2 0 0 3 4 4 7
3 0 0 3 4 5 7
4 0 0 3 4 5 7
7-4=3-3=0

Example

Let's run our algorithm on the following data:

n = 4 (# of elements)
W = 5 (max weight)
Elements (weight, benefit):
(2,3), (3,4), (4,5), (5,6)

Example(2)

$$B[k,w] = \begin{cases} B[k-1,w] & \text{if } w_k > w \\ \max\{B[k-1,w], B[k-1,w-w_k]+b_k\} & \text{else} \end{cases}$$

W \ i	0	1	2	3	4
0	0				
1	0				
2	0				
3	0				
4	0				
5	0				

for w = 0 to W

B[0,w] = 0

Example (3)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0				
2	0				
3	0				
4	0				
5	0				

for i = 0 to n

B[i,0] = 0

Example (4)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0 →	0			
2	0				
3	0				
4	0				
5	0				

$i=1$

$b_i=3$

$w_i=2$

$w=1$

$w-w_i=-1$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (5)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0			
2	0	**3**			
3	0				
4	0				
5	0				

$i=1$

$b_i=3$

$w_i=2$

$w=2$

$w-w_i=0$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (6)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0			
2	0	3			
3	0	**3**			
4	0				
5	0				

$i=1$

$b_i=3$

$w_i=2$

$w=3$

$w-w_i=1$

if w_i <= w // item i can be part of the solution
  if $b_i + B[i-1,w-w_i] > B[i-1,w]$
    $B[i,w] = b_i + B[i-1,w- w_i]$
  else
    B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w] // $w_i > w$

Example (7)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0			
2	0	3			
3	0	3			
4	0	**3**			
5	0				

$i=1$

$b_i=3$

$w_i=2$

$w=4$

$w-w_i=2$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (8)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0			
2	0	3			
3	0	3			
4	0	3			
5	0	**3**			

$i=1$

$b_i=3$

$w_i=2$

$w=5$

$w-w_i=2$

if w_i <= w // item i can be part of the solution
  if $b_i + B[i-1,w-w_i] > B[i-1,w]$
    $B[i,w] = b_i + B[i-1,w- w_i]$
  else
    B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w] // $w_i > w$

Example (9)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	→ **0**		
2	0	3			
3	0	3			
4	0	3			
5	0	3			

$i=2$

$b_i=4$

$w_i=3$

$w=1$

$w-w_i=-2$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (10)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0		
2	0	3	**3**		
3	0	3			
4	0	3			
5	0	3			

$i=2$

$b_i=4$

$w_i=3$

$w=2$

$w-w_i=-1$

```
if w_i <= w // item i can be part of the solution
    if b_i + B[i-1,w-w_i] > B[i-1,w]
        B[i,w] = b_i + B[i-1,w- w_i]
    else
        B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (11)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0		
2	0	3	3		
3	0	3	**4**		
4	0	3			
5	0	3			

$i=2$

$b_i=4$

$w_i=3$

$w=3$

$w-w_i=0$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (12)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0		
2	0	3	3		
3	0	3	4		
4	0	3	**4**		
5	0	3			

$i=2$

$b_i=4$

$w_i=3$

$w=4$

$w-w_i=1$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (13)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0		
2	0	3	3		
3	0	3	4		
4	0	3	4		
5	0	3	7		

$i=2$

$b_i=4$

$w_i=3$

$w=5$

$w-w_i=2$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (14)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0 →	**0**	
2	0	3	3 →	**3**	
3	0	3	4 →	**4**	
4	0	3	4		
5	0	3	7		

$i=3$

$b_i=5$

$w_i=4$

$w=1..3$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (15)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	
2	0	3	3	3	
3	0	3	4	4	
4	0	3	4	**5**	
5	0	3	7		

$i=3$

$b_i=5$

$w_i=4$

$w=4$

$w- w_i=0$

if $w_i <= w$ // item i can be part of the solution
  if $b_i + B[i-1,w-w_i] > B[i-1,w]$
    $B[i,w] = b_i + B[i-1,w- w_i]$
  else
    $B[i,w] = B[i-1,w]$
else $B[i,w] = B[i-1,w]$ // $w_i > w$

Example (15)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	
2	0	3	3	3	
3	0	3	4	4	
4	0	3	4	5	
5	0	3	7 →	7	

$i=3$

$b_i=5$

$w_i=4$

$w=5$

$w- w_i=1$

```
if w_i <= w // item i can be part of the solution
    if b_i + B[i-1,w-w_i] > B[i-1,w]
        B[i,w] = b_i + B[i-1,w- w_i]
    else
        B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (16)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0 →	**0**
2	0	3	3	3 →	**3**
3	0	3	4	4 →	**4**
4	0	3	4	5 →	**5**
5	0	3	7	7	

$i=3$

$b_i=5$

$w_i=4$

$w=1..4$

```
if w_i <= w // item i can be part of the solution
   if b_i + B[i-1,w-w_i] > B[i-1,w]
      B[i,w] = b_i + B[i-1,w- w_i]
   else
      B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Example (17)

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	3	3	3	3
3	0	3	4	4	4
4	0	3	4	5	5
5	0	3	7	7 →	7

$i=3$

$b_i=5$

$w_i=4$

$w=5$

```
if w_i <= w // item i can be part of the solution
    if b_i + B[i-1,w-w_i] > B[i-1,w]
        B[i,w] = b_i + B[i-1,w- w_i]
    else
        B[i,w] = B[i-1,w]
else B[i,w] = B[i-1,w]  // w_i > w
```

Comments *

- This algorithm only finds the max possible value that can be carried in the knapsack
- To know the items that make this maximum value, an addition to this algorithm is necessary
- see LCS algorithm from the textbook to for extract this data from the table we built

How to find actual Knapsack Items

```
i=n, k=W   // i for the columns, k for rows
while (i or k > 0)
        if (V[i,k] != V[i-1,k]) then
                ith item = 1 //include it in the knapsack
                i = i-1, k = k-w_i
        else
                ith item = 0 //don't include it in the knapsack
                i = i-1
```

Finding The Items

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	3	3	3	3
3	0	3	4	4	4
4	0	3	4	5	5
5	0	3	7	7 ←	7

i=4

k=5

V[i,k]=7

V[i-1,k]=7

$x_4=0$

Finding The Items

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	3	3	3	3
3	0	3	4	4	4
4	0	3	4	5	5
5	0	3	7 ← 7		**7**

i=3

k=5

V[i,k]=7

V[i-1,k]=7

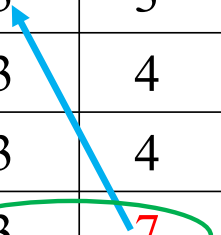
$x_4=0$, $x_3=0$

Finding The Items

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	3	3	3	3
3	0	3	4	4	4
4	0	3	4	5	5
5	0	3	7	7	7



i=2
k=5
w_i=3
b_i=4
V[i,k]=7
V[i-1,k]=3

x_4=0, x_3=0, x_2=1 and k=k-w_i = 5-3=2

Next loop: i=1 and k=2

Finding The Items

Items:

1: (2,3)
2: (3,4)
3: (4,5)
4: (5,6)

W \ i	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	3	3	3	3
3	0	3	4	4	4
4	0	3	4	5	5
5	0	3	7	7	**7**

i=1
k=2
w_i=2
b_i=3
V[i,k]=3
V[i-1,k]=0

x_4=0, x_3=0, x_2=1, x_1=1 and k=k-w_i = 2-2=0
Next loop: i=0 and k=0 stop
The optimal knapsack = {1,2}

Examples of Dynamic Programming Algorithms

- Computing binomial coefficients
- Coin Change problem
- Floyd's algorithms for all-pairs shortest paths
- Constructing an optimal binary search tree
- Some instances of difficult discrete optimization problems: knapsack

Computing Binomial Coefficients

(Read it)

- A binomial coefficient, denoted *C(n, k),* is the number of combinations of *k* elements from an *n*-element set *(0 ≤ k ≤ n).*
- Recurrence relation (a problem ☐ 2 overlapping problems)
  - C(n, k) = C(n-1, k-1) + C(n-1, k), for n > k > 0, and
  - C(n, 0) = C(n, n) = 1
- Dynamic programming solution:
  - *Record the values of the binomial coefficients in a table of* n+1 *rows and* k+1 *columns, numbered from* 0 *to* n *and* 0 *to* k *respectively.*

1					
1	1				
1	2	1			
1	3	3	1		
1	4	6	4	1	
1	5	10	10	5	1

Making Change Problem

DP for Change Problem

C: Minimum number of coins

P: Amount to change

di: coin i

$$C[p] = \begin{cases} 0 & if\ p = 0 \\ \min_{i:d_i \leq p}\{1 + C[p - d_i]\} & if\ p > 0 \end{cases}$$

DP for Change Problem

- **Change(d, k, n)** // d: set of coins, n: total amount, k: total number of coins

```
C[0] ← 0
for p ← 1 to n
    min ← ∞
    for i ← 1 to k
        if d[i] ≤ p then
            if 1 + C[p − d[i]] < min then
            min ← 1 + C[p − d[i]]
            coin ← i
    C[p] ← min
    S[p] ← coin
return C and S
```

Coin Change Problem
Example

- Amount P= 5 coins d = {1,2,3}

	0	1	2	3	4	5
c	0	1	1	1	2	2

	0	1	2	3	4	5
s	0	1	2	3	1	2

- Solution:
  - *Minimum No. of coins = 2*
  - *Set of coins are = {2, 3}*
  - *5-2 = 3*
  - *3-3 = 0*

General Comments

- The crucial step in designing a dynamic programming algorithm:
  - *Deriving a recurrence relating a solution to the problem's current instance with solutions of its smaller (and overlapping) subinstances.*

When Can Dynamic Programming Be Used?

- Dynamic programming can be used to solve ***optimization problems***, that is, problems where many solutions exist, but we are looking for an optimal solution
- **Two properties have to be present:**
- **Optimal substructure:**
  - *The optimal solution for the current problem instance contains optimal solutions for smaller problem instances.*
- **Overlapping subproblems:**
  - *Many subproblems share subproblems.*

Properties of a problem that can be solved with dynamic programming

- Simple Subproblems
  - *We should be able to break the original problem to smaller subproblems that have the same structure*
- Optimal Substructure of the problems
  - *The solution to the problem must be a composition of subproblem solutions*
- Subproblem Overlap
  - *Optimal subproblems to unrelated problems can contain subproblems in common*

Review: Dynamic programming

- DP is a method for solving certain kind of problems
- DP can be applied when the solution of a problem includes solutions to subproblems
- We need to find a recursive formula for the solution
- We can recursively solve subproblems, starting from the trivial case, and save their solutions in memory
- In the end we'll get the solution of the whole problem